

CSCI1409 Deep Learning | Aidyn Abirov | Week 4 Tasks 1-3 - Estimated time: about 1.5 hours – **Deadline:** before the next lecture – **Submit:** notebook via MS Teams Assignments

This homework is about **gradient descent (GD)**: seeing how it moves step by step, what the learning rate does, and how to draw the path it takes on a loss surface.

Rules

- Work individually.
- **Tasks 1-2: no AI** (search engines, lecture notes, textbooks are fine – list references). Do not use AI-generated search summaries.
- **Task 3: AI allowed** only to explain concepts or help write/debug Task 3 code – not Tasks 1-2. State which tool you used and for what. You must understand and be able to defend every submitted line.
- I or the TA may ask you to explain, trace, or modify code without AI during the practice defense.
- Use Python, NumPy, and Matplotlib only – no autodiff, framework optimizers, or scikit-learn estimators.
- Submit Surname_Firstname_Homework04.ipynb via the Assignments tab on Teams; restart the kernel and run all cells first. No submissions once the next lecture starts.

The GD update. Compute the gradient g at the current point and take a small step the opposite way: $w \leftarrow w - \eta g$, where η is the learning rate.

Task 1 - GD on a curve with two valleys

About 25 minutes. No AI. A calculator is fine.

Minimize $L(w) = (w^2 - 1)^2$. It has two valleys (minima) at $w = -1$ and $w = +1$, and a bump at $w = 0$. Its gradient is $g(w) = 4w^3 - 4w$.

Do:

1. With $\eta = 0.1$ and $w_0 = 0.5$, do 2 updates by hand. Table: step, w , g , new w , new L .
2. Before computing anything, predict what happens with the same η from $w_0 = -0.5$. Then check your prediction with one update.
3. From $w_0 = 1.5$, the gradient is $g = 7.5$. Do 1 update by hand for each $\eta \in \{0.1, 0.2, 0.24\}$ and mark where each lands: in the right valley, on the bump, or past the bump?
4. In NumPy, run 30 updates for each of the five runs above (items 1-3). Draw each run on its own copy of the curve (one figure, one panel per run), with the points connected in order.

```
w_grid = np.linspace(-1.8, 1.8, 200)
plt.plot(w_grid, (w_grid**2 - 1)**2, color="lightgray")
plt.plot(points, (np.array(points)**2 - 1)**2, "o-") # points = [w0, w1, ..., w30]
plt.ylim(-0.2, 2.5); plt.xlabel("w"); plt.ylabel("L(w)"); plt.show()
```

Explain:

- Which valley does each run end in? What decides it?
- The run with $\eta = 0.2$ stops at $w = 0$. Is that a minimum? What does this tell you about points where the gradient is zero?
- Where on the curve is the gradient largest, and where is it zero? Use your plots to answer.

Sanity check: $g(0.5) = -1.5$, so the first update in item 1 gives $w = 0.65$.

Task 2 - GD on a surface

About 25 minutes. No AI.

Use $L(w_1, w_2) = w_1^2 + 10 w_2^2$ starting from $(2, 1)$. The gradient is $g = (2w_1, 20w_2)$. The minimum is at $(0, 0)$.

Do:

1. By hand, do 2 updates with $\eta = 0.05$ and 1 update with $\eta = 0.09$. Write the points and the losses.
2. Draw a contour plot of L (code below). At the start $(2, 1)$, draw two arrows: the direction GD moves in, $-g$, and the direction straight to the minimum. Do they point the same way?
3. In NumPy, run 30 updates for each $\eta \in \{0.01, 0.05, 0.09, 0.11\}$. Draw each path on the contour plot (one figure, four panels) and plot the four loss curves in a second figure with a log y-axis.

```
w1, w2 = np.meshgrid(np.linspace(-2.5, 2.5, 200), np.linspace(-1.5, 1.5, 200))
plt.contour(w1, w2, w1**2 + 10 * w2**2, levels=15, colors="lightgray")
plt.plot(path[:, 0], path[:, 1], "o-", markersize=3) # path has shape (31, 2)
plt.xlim(-2.5, 2.5); plt.ylim(-1.5, 1.5); plt.show()
```

Explain: Why does the path not head straight to the minimum? Which learning rate is too slow, which zig-zags, and which blows up? In which direction, w_1 or w_2 , does the zig-zag happen, and why that one?

Sanity check: after the first update with $\eta = 0.05$, the point is $(1.8, 0)$.

Task 3 - Watch GD fit a line

About 40 minutes. AI assistance allowed, with individual defense.

Fit the line $\hat{y} = wx + b$ to five points with MSE loss, starting from $w = 0, b = 0$:

```
x = np.array([0., 1., 2., 3., 4.])
y = np.array([1., 3., 5., 7., 9.])
```

$$L(w, b) = \frac{1}{N} \sum_i (\hat{y}_i - y_i)^2, \quad \frac{\partial L}{\partial w} = \frac{2}{N} \sum_i (\hat{y}_i - y_i) x_i, \quad \frac{\partial L}{\partial b} = \frac{2}{N} \sum_i (\hat{y}_i - y_i)$$

A. Code. Write `mse(w, b)`, `gradients(w, b)`, and `gd(lr, n_updates)`. `gd` starts from $(0, 0)$ and returns `path`, an array of shape $(n_updates + 1, 2)$ holding (w, b) before the first update and after every update, and `history`, the matching $n_updates + 1$ losses. Print the loss at the start.

Sanity check: the starting loss is 33. With $lr = 0.05$, the first update gives $(w, b) = (1.4, 0.5)$ and loss 3.61.

B. The line learning. Run `gd(0.05, 100)`. In one figure, plot the five data points and the model's line after updates 0, 1, 2, 5, 20, and 100, with a legend.

C. The path on the loss surface. Draw a contour plot of $L(w, b)$ for $w \in [-0.5, 4.5]$, $b \in [-0.5, 2.5]$ and mark the minimum $(2, 1)$. Run 100 updates for each lr in $[0.01, 0.05, 0.14]$ and draw each path on its own copy of the contour plot (one figure, three panels). Then plot the loss curves for lr in $[0.01, 0.05, 0.14, 0.16]$ in one figure with a log y-axis.

Questions (Markdown cells):

1. In B, which parameter gets close to its correct value first, w or b ? Use the shape of the contours in C to explain why.

2. Describe the path for $\text{lr} = 0.14$. It looks wild at first — does it end up better or worse than $\text{lr} = 0.05$ after 100 updates? What happens with $\text{lr} = 0.16$?
3. If you had a new problem and no idea which learning rate to use, how would you choose one using what you saw here?

In the defense, be ready to explain any line of `gd`, why `path` and `history` have one more entry than the number of updates, and what changes in your plots if you double the learning rate.

Optional (not required): train the Week 3 network from Homework 3 with GD ($\eta = 0.05$) and plot its prediction curve on $[-2, 2]$ after 0, 10, 50, and 300 updates, together with the target $|x|$.

Submission checklist

- All three tasks complete, with explanations and visible outputs
- Task 1: hand calculations, the five curve plots, answers to the three questions
- Task 2: hand updates, contour plot with the two arrows, four paths, loss curves
- Task 3: sanity values match; line-fitting plot; contour plot with three paths; loss curves; answers to the three questions
- External references and any Task 3 AI use recorded; no AI used for Tasks 1–2
- Notebook runs from a fresh kernel; submitted via Teams Assignments before the next lecture begins

Reminder: a working notebook does not replace the individual defense during practice.